

Object Oriented Programming – Homework

Q1: write the Circle class according to the specifications below:

```
class Circle():
    def __init__(self):
        # initilizes self with radius
        pass

    def get_radius(self):
        # returns radius of self
        pass

    def set_radius(self, radius):
        # changes the radius attribute of self to radius
        pass

    def area(self):
        # computes and returns area of self
        pass

    def __eq__(self, other):
        # other is a Circle object
        # returns True if self and other has the same radius value
        pass

    def __gt__(self, other):
        # other is a Circle object
        # returns self or other, Circle object with the bigger radius
        pass

    def __add__(self, other):
        # other is a Circle object
        # returns a new Circle object that it's radius is
        # the sum of self and other's radius
        pass

    def __str__(self):
        # a Circle's string representation is the radius
        pass
```

Q2: Create a BankAccount class with the following **three types of attributes**:

Attribute	Type	Accessibility	Description
account_holder	Public	Anywhere	Name of the account holder
_transaction_history	Protected	Class + subclasses	List to store all transactions
__balance	Private	Only inside class	Current balance (cannot access directly)

Requirements:

1. Constructor (__init__)

- Takes account_holder (public) as parameter
- Initializes __balance to 0 (private)
- Initializes _transaction_history as an empty list (protected)

2. Methods:

- deposit(amount) – adds amount to balance, records transaction
- withdraw(amount) – subtracts if sufficient funds, otherwise shows error
- check_balance() – returns current balance (but does NOT allow direct modification)
- show_history() – prints all transactions (protected method, but callable from outside)

3. Business rules:

- Cannot withdraw more than current balance
- All deposits/withdrawals should be recorded in _transaction_history
- Transactions should be stored as strings like: "Deposited \$100" or "Withdrew \$30"

4. **Demonstration** – after creating the class, run this code:

```
# Create account
account = BankAccount("John")

# Test all methods
account.check_balance()    # Balance: $0
account.deposit(100)       # Deposited $100
account.withdraw(30)       # Withdrew $30
account.withdraw(100)      # Insufficient funds!
account.check_balance()    # Balance: $70
account.show_history()     # Show all transactions
```

1. Try to access `__balance` directly from outside the class. What happens? Why?
2. Try to modify `account_holder` from outside. Is it possible? Should it be?
3. Create a subclass `SavingsAccount` that inherits from `BankAccount`. Can it access `__transaction_history`? Can it access `__balance`?
4. Add a **property decorator (@property)** for `balance` that allows reading but not setting (read-only).